



# 6. OOPs Concepts in C++

## 6.1 Class and Object

## 6.2 Constructor and Destructor

## 6.3 Programs in CodeBlocks

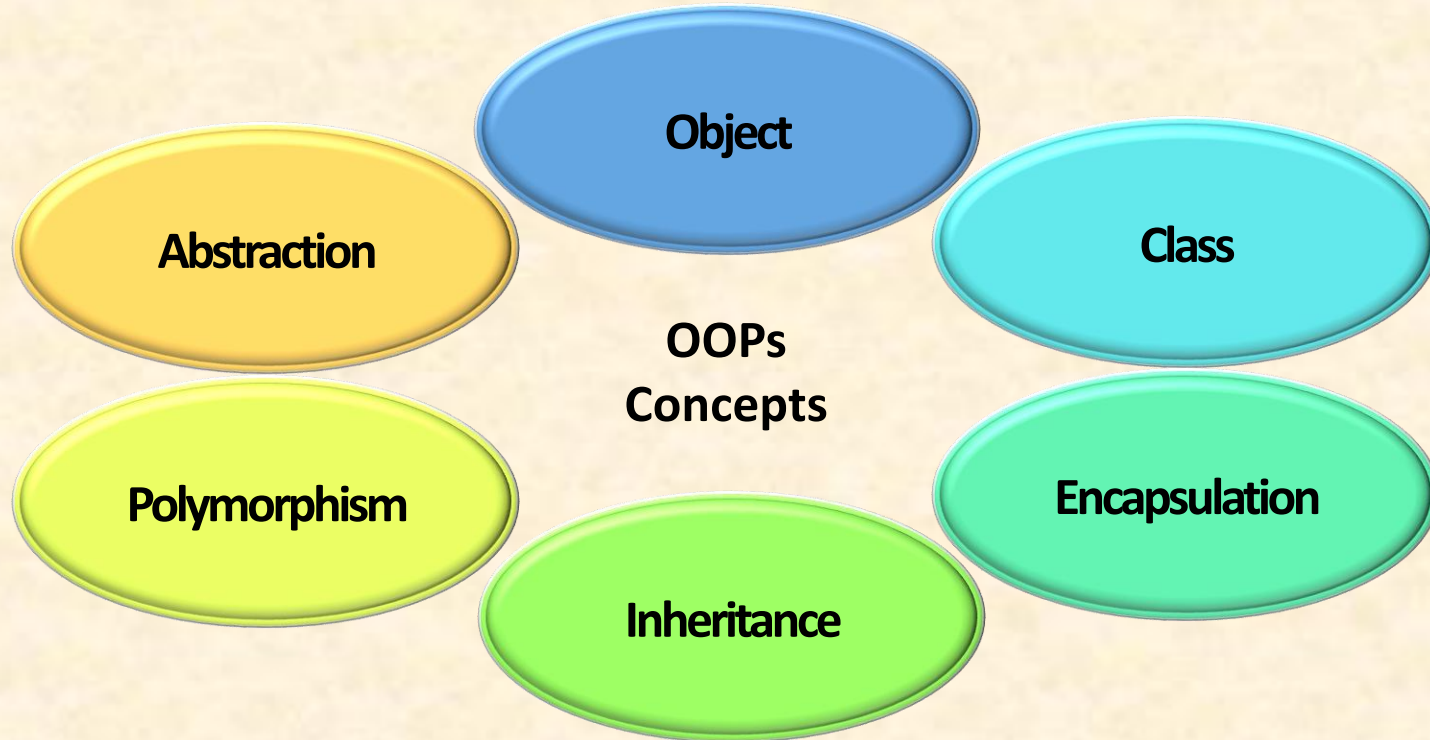
# 6.1 Class and Object



1. OOPs Concepts
2. Class
3. Object

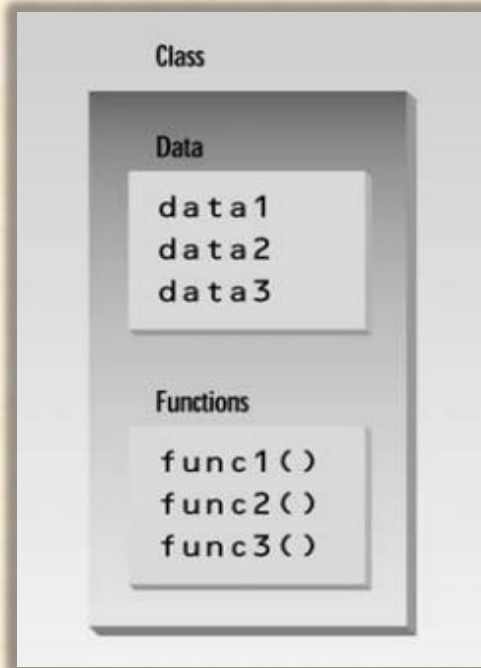


- To understand object-oriented concepts
- To make use of objects and classes for developing programs



## 6.1 Class and Object

### C++ Class



- A class is a user-defined data type that serves as a blueprint for creating objects.
  - It defines the data and functions that belong to those objects.
- 
- **Encapsulation** is the process of combining data and the functions that operate on that data into a class and hiding the data from direct access using access specifiers.
  - It protects the data from unauthorized or accidental modification.

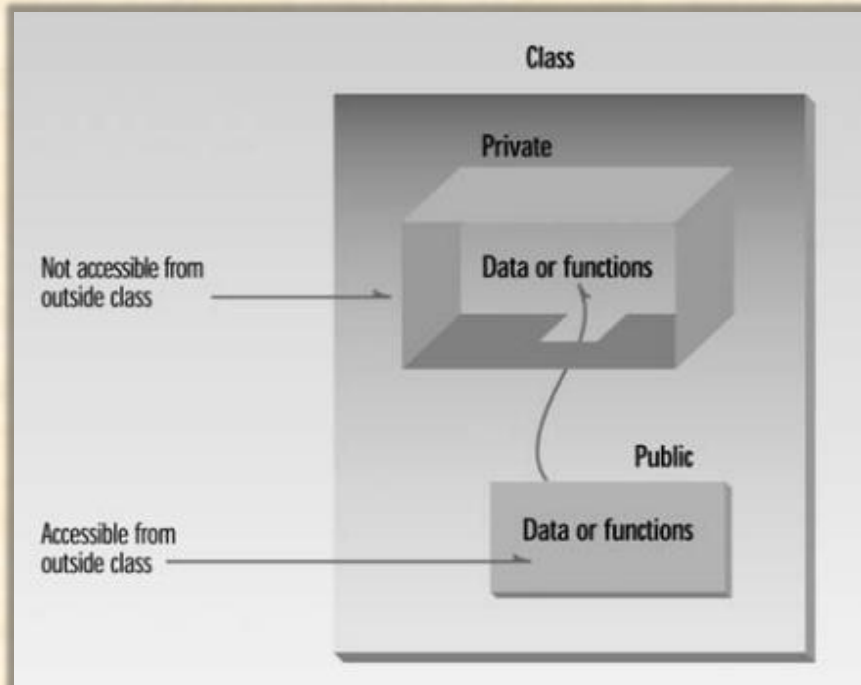
### C++ Class Examples

|                   |                           |                                         |
|-------------------|---------------------------|-----------------------------------------|
| <b>Class name</b> | <b>Student</b>            | <b>Circle</b>                           |
| <b>Data</b>       | name<br>grade             | radius<br>color                         |
| <b>Functions</b>  | getName()<br>printGrade() | getRadius()<br>getArea()<br>showColor() |

### Defining the Class

```
class SmallObj //define a class
{
    private:
        int data; //class data
    public:
        void setData(int d) //member function to set data
        {
            data = d;
        }
        void showData() //member function to display data
        {
            cout<<"Data is " <<data<< endl;
        }
};
```

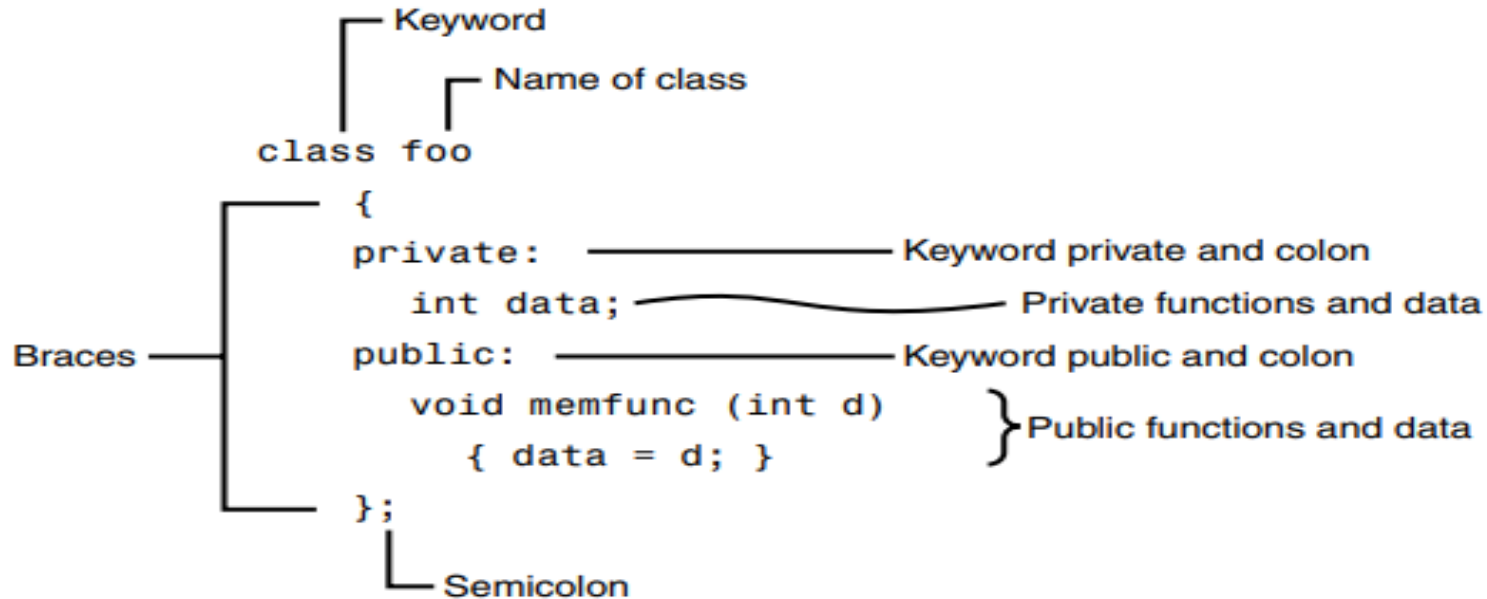
### Private and Public



- A key feature of object-oriented programming is **data hiding**.
- In C++, the default access specifier is **private**.

## 6.1 Class and Object

### Functions Are Public, Data Is Private



```
class foo
{
private:
    int data;
public:
    void memfunc (int d)
    { data = d; }
};
```

The diagram illustrates the syntax of a C++ class definition with the following annotations:

- Keyword:** Points to the word `class`.
- Name of class:** Points to the identifier `foo`.
- Braces:** A bracket on the left side of the opening curly brace `{` and the closing curly brace `}`.
- Keyword private and colon:** Points to the `private:` access specifier.
- Private functions and data:** A bracket on the right side of the `private` block, encompassing `int data;`.
- Keyword public and colon:** Points to the `public:` access specifier.
- Public functions and data:** A bracket on the right side of the `public` block, encompassing the `void memfunc (int d) { data = d; }` function.
- Semicolon:** Points to the terminating semicolon `;` at the end of the class definition.

### Getters and Setters

#### Getter (Accessor):

- A **getter** is a public member function that returns the value of a private data member.
- It provides controlled read access to the data.

#### Syntax

```
Type getVariable()  
{  
    return variable;  
}
```

### Getters and Setters

#### Setter (Mutator):

- A **setter** is a public member function that modifies the value of a private data member.
- It provides controlled write access to the data and can validate input before updating the member.

#### Syntax

```
void setVariable(Type value)
{
    variable = value;
}
```

### Getters and Setters

```
class Book //define a class
{
    private:
        int page; //class data
    public:
        void setPage(int p) //setter for page
        {
            page = p;
        }
        int getPage() //getter for page
        {
            return page;
        }
};
```

### Keyword "this"

- The **this** keyword is a pointer that refers to the current object that invokes (calls) a member function.

#### Syntax

`this -> memberName`    **or**    `(*this).memberName`

### Keyword "this"

```
class Book //define a class
{
    private:
        int page; //class data
    public:
        void setPage(int page) //setter for page
        {
            this->page=page;
        }
};
```

### C++ Objects

- An object is an instance of a class.
- When an object is created, memory is allocated for its data members.
- Each object has its own copy of the data members, while all objects share the same member function definitions.
- Objects are used to access the members of a class.

#### Syntax

```
int main()  
{  
    ClassName ObjectName;  
}
```

### A Simple Class

```
#include <iostream>
using namespace std;
class SmallObj
{
    private:
        int data;
    public:
        void setData(int d)
        { data = d; }

        void showData()
        { cout<<"Data is "
          <<data<< endl;
        }
};
```

```
int main()
{
    SmallObj s1, s2;
    s1.setData(1066);
    s2.setData(1776);
    s1.showData();
    s2.showData();
    return 0;
}
```

Output

```
Data is 1066
Data is 1776
```

## 6.1 Class and Object

### Defining Objects

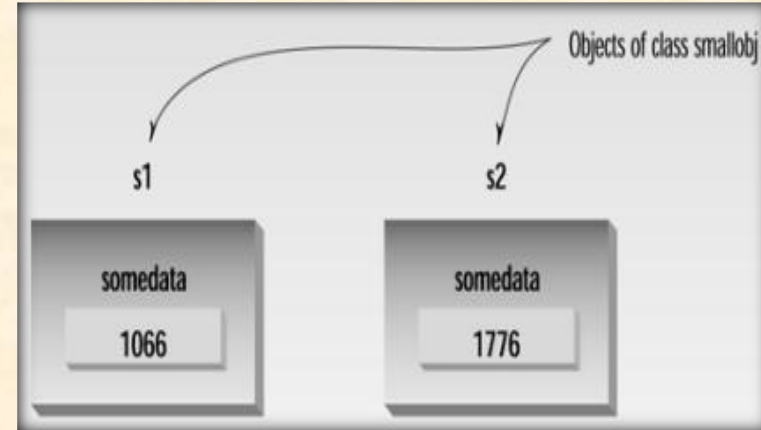
```
SmallObj s1, s2;
```

### Accessing/Calling Member Functions

```
s1.setData(1066);  
s2.setData(1776);
```

### Messages

```
s1.showData();  
s2.showData();
```



### Accessing Member Data and Function

```
#include <iostream>
using namespace std;
class Person
{ public:
    string name;
    void displayName()
    { cout<<"Name is "<<name; }
};
int main()
{
    Person p;
    p.name = "Mya Mya"; //accessing data member
    p.displayName();    //accessing member function
    return 0;
}
```

### Member Function outside the Class

```
return_type ClassName::FunctionName()  
{ }
```

```
#include <iostream>  
using namespace std;  
class Person  
{ public:  string name;  
          void displayName();  
};  
void Person::displayName()  
{  cout<<"Name is "<<name;  }  
int main()  
{  Person p;  
    p.name = "Mya Mya";  p.displayName();  
    return 0;  
}
```

### Creating and Using Objects

```
#include <iostream>
using namespace std;
class Distance
{   private:
        int feet;    float inches;
    public:
        void setdist(int ft, float in)
        { feet = ft;  inches = in; }
        void getdist()
        {   cout << "Enter feet: ";   cin >> feet;
            cout << "Enter inches: "; cin >> inches;
        }
        void showdist()
        { cout << feet << "'-" << inches << "'"; }
};
```

```
int main()
{   Distance dist1, dist2;
    dist1.setdist(11, 6.25);
    dist2.getdist();
    cout << "\ndist1 = ";
    dist1.showdist();
    cout << "\ndist2 = ";
    dist2.showdist();
    cout << endl;
    return 0;
}
```

Output

```
Enter feet: 6
Enter inches: 2.5

dist1 = 11'-6.25"
dist2 = 6'-2.5"
```



### Quiz 1

OOP stands for?

- A. Object to Object Programming
- B. Object Oriented Programming
- C. Online Objects Programming
- D. Object Oriented Processes

Answer is B

### Quiz 2

Which of the following is a valid class declaration?

- A. `class A { int x; };`
- B. `class A { }`
- C. `public class A { }`
- D. `object A { int x; }`

Answer is A

### Quiz 3

The mechanism by which the data and functions are bound together within an object is called as \_\_\_\_\_.

- A. Overriding
- B. Polymorphism
- C. Overloading
- D. Encapsulation

Answer is D

### Let's summarize what we learned today

- A **class** is a user defined blueprint or prototype from which **objects** are created.
- It represents the set of properties or methods that are common to all **objects** of one type.
- Classes are designed for code reuse.



# 6. OOPs Concepts in C++

6.1 Class and Object

**6.2 Constructor and Destructor**

6.3 Programs in CodeBlocks

# 6.2 Constructor and Destructor



1. Constructor and Destructor
2. Objects as Function Arguments
3. Returning Objects from Functions
4. Arrays as Class Member Data
5. Arrays of Objects



- To understand about constructor and destructor
- To apply the objects as function arguments and return data types
- To use array in class and object

### Constructor

- A constructor is a special member function that is **executed automatically** when ever an object is created.
- **Same name** as the class
- **No return type** is used for constructor
- Used to **initialize data members**.
- Can be **overloaded**.

## 6.2 Constructor and Destructor

```
class Distance
{
    private:  int feet;    float inches;
};
```

### Default Constructor (No Arguments Constructor)

```
Distance()
{  feet=0;  inches=0.0; }
```

**OR**

```
Distance() : feet(0), inches(0.0)
{ }
```

## 6.2 Constructor and Destructor

```
class Distance
{
    private:  int feet;    float inches;
};
```

### Parameterized Constructor (Arguments Constructor)

```
Distance(int ft, float in)
{   feet=ft;   inches=in;   }
```

OR

```
Distance(int feet, float inches)
{   this->feet=feet;   this->inches=inches; }
```

OR

```
Distance(int ft, float in) : feet(ft), inches(in)
{ }
```

### Overloaded Constructors

```
//no argument constructor
```

```
Distance() : feet(0), inches(0.0)
```

```
{ }
```

```
//two arguments constructor
```

```
Distance(int ft, float in) : feet(ft), inches(in)
```

```
{ }
```

### Overloaded Constructors: A Counter Example

```
#include <iostream>
using namespace std;
class Counter
{   private:
    int count;
    public:
    Counter() : count(0)
    {}
    Counter(int c) : count(c)
    { }
    void inc_count()
    {   count++;   }
    int get_count()
    {   return count; }
};
```

```
int main()
{   Counter c1;           //call default constructor
    Counter c2(100);      //call parameterized constructor
    cout << "\nc1=" << c1.get_count();
    cout << "\nc2=" << c2.get_count();
    c1.inc_count();
    c2.inc_count();
    cout << "\nc1=" << c1.get_count();
    cout << "\nc2=" << c2.get_count();
    return 0;
}
```

Output

```
c1=0
c2=100
c1=1
c2=102
```

### Copy Constructor

- A copy constructor is a special constructor that creates a new object by initializing it with another object of the same class.

#### Syntax

```
ClassName(const ClassName &object)
{
    // Copy data members
}
```

## 6.2 Constructor and Destructor

### Copy Constructor: Distance Example

```
#include <iostream>
using namespace std;
class Distance //English Distance class
{ private:
    int feet;    float inches;
public:
    Distance() : feet(0), inches(0.0) { } // Default constructor
    Distance(int ft, float in):feet(ft),inches(in){ }
    Distance(const Distance &d) { // Copy constructor
        feet = d.feet; inches = d.inches;
    }
    void getdist() {
        cout << "\nEnter feet: "; cin >> feet;
        cout << "Enter inches: "; cin >> inches;
    }
    void showdist()
    { cout << feet << "'-" << inches << "'"; }
};
```

```
int main()
{
    Distance dist1(11, 6.25);
    // Calls copy constructor
    Distance dist2(dist1);
    // Also calls copy constructor
    Distance dist3 = dist1;
    cout << "\ndist1 = ";
    dist1.showdist();
    cout << "\ndist2 = ";
    dist2.showdist();
    cout << "\ndist3 = ";
    dist3.showdist();
    cout << endl;
    return 0;
}
```

**Output**

```
dist1 = 11'-6.25"
dist2 = 11'-6.25"
dist3 = 11'-6.25"
```

### Compiler-Generated Copy Constructor

```
#include <iostream>
using namespace std;
class Distance //English Distance class
{ private:
    int feet;    float inches;
public:
    Distance() : feet(0), inches(0.0)    { }
    Distance(int ft, float in):feet(ft),inches(in){ }
    void getdist() {
        cout << "\nEnter feet: "; cin >> feet;
        cout << "Enter inches: "; cin >> inches;
    }
    void showdist()
    { cout << feet << "'-" << inches << "'"; }
};
```

```
int main()
{
    Distance dist1(11, 6.25);
    Distance dist2(dist1);
    Distance dist3 = dist1;
    cout << "\ndist1 = ";
    dist1.showdist();
    cout << "\ndist2 = ";
    dist2.showdist();
    cout << "\ndist3 = ";
    dist3.showdist();
    cout << endl;
    return 0;
}
```

Output

```
dist1 = 11'-6.25"
dist2 = 11'-6.25"
dist3 = 11'-6.25"
```

### Destructor

- A destructor **is called automatically when an object is destroyed.**
- Same name as the class.
- Begins with the tilde (~).
- Destructor take no argument and have no return value.
- Cannot be overloaded (only one destructor per class).

### Destructor Example

```
#include <iostream>
using namespace std;
class Foo
{   public:   Foo()    // constructor
                {   cout<<"Start Program\n"; }

                void show()
                {   cout<<"This is show function.\n"; }

                ~Foo() //destructor
                {   cout<<"End Program"; }
};

int main()
{   Foo f;
    f.show();
    return 0;
}
```

```
Start Program
This is show function.
End Program
```

## 6.2 Constructor and Destructor

### Objects as Function Arguments

```
#include <iostream>
using namespace std;
class Distance
{private: int feet; float inches;
public:
    Distance() : feet(0), inches(0.0){ }
    Distance(int ft,float in):feet(ft),inches(in)
    { }
    void getdist() {
        cout << "\nEnter feet: "; cin >> feet;
        cout << "Enter inches: "; cin >> inches;
    }
    void showdist(){
        cout << feet << "'-" << inches << "'\n";
    }
    void add_dist( Distance, Distance );
};
```

```
void Distance::add_dist(Distance d1,Distance d2)
{   inches = d1.inches + d2.inches;
    feet = 0;
    if(inches >= 12.0) {inches -= 12.0; feet++;}
    feet += d1.feet + d2.feet;
}

int main()
{   Distance dist1, dist3;
    Distance dist2(11, 6.25);
    dist1.getdist();
    dist3.add_dist(dist1, dist2);
    cout << "\ndist1 = "; dist1.showdist();
    cout << "\ndist2 = "; dist2.showdist();
    cout << "\ndist3 = "; dist3.showdist();
    return 0;
}
```

```
Enter feet: 6
Enter inches: 3.5
```

```
dist1 = 6'-3.5"
dist2 = 11'-6.25"
dist3 = 17'-9.75"
```

## Returning Objects from Functions

```
#include <iostream>
using namespace std;
class Distance
{ private:  int feet;    float inches;
  public:
    Distance() : feet(0), inches(0.0)          { }
    Distance(int ft, float in):feet(ft),inches(in)
    { }
    void getdist(){
        cout << "\nEnter feet: "; cin >> feet;
        cout << "Enter inches: "; cin >> inches;
    }
    void showdist()
    { cout << feet << "\'-" << inches << '\\"'; }
    Distance add_dist(Distance);
};
```

```
Distance Distance::add_dist(Distance d2)
{   Distance temp;
    temp.inches = inches + d2.inches;
    if(temp.inches >= 12.0)
    { temp.inches -= 12.0; temp.feet = 1; }
    temp.feet += feet + d2.feet;
    return temp;  }

int main()
{   Distance dist1, dist3;
    Distance dist2(11, 6.25);
    dist1.getdist();
    dist3 = dist1.add_dist(dist2);
    cout << "\ndist1 = "; dist1.showdist();
    cout << "\ndist2 = "; dist2.showdist();
    cout << "\ndist3 = "; dist3.showdist();
    return 0;    }
```

### const Member Functions and const Objects

#### Const Member Function

- A const member function is a member function declared with the **const** keyword after the parameter list.
- It promises not to modify the object's data members.

#### Syntax:

```
returnType functionName(parameters) const
{
    // cannot modify data members
}
```

#### Const Objects

- A const object is an object declared with the **const** keyword.
- It cannot be modified and can call only const member functions.

#### Syntax:

```
const ClassName objectName;
```

### const Member Functions and const Objects

```
#include <iostream>
using namespace std;
class Distance {
    private: int feet;    float inches;
    public: Distance(int ft, float in) : feet(ft), inches(in){ }
        void getdist(){
            cout << "\nEnter feet: "; cin >> feet;
            cout << "Enter inches: "; cin >> inches;
        }
        void showdist() const
        { cout << feet << "'-" << inches << "'"; }
};

int main()
{
    const Distance football(300, 0);
    // football.getdist(); //ERROR: getdist() not const
    cout << "football = ";    football.showdist();
    return 0;    }
```

### Static Data Member

- Declared using the static keyword.
- Belongs to the class, not to objects.
- Only one copy exists.
- Shared by all objects.
- Memory is allocated only once.
- Must be defined outside the class.

#### Syntax:

```
class ClassName
{
    private:
        static int staticVar;
};

int ClassName :: staticVar = 0;
```

### Example of Static Data Member

```
#include <iostream>
using namespace std;
class Counter{
    private: static int count;
    public: Counter(){
        count++;
    }
    int getCount(){ return count; }
};
int Counter::count = 0
int main()
{
    Counter c1, c2, c3;
    cout << "count is " << c1.getCount() << endl;
    cout << "count is " << c2.getCount() << endl;
    cout << "count is " << c3.getCount() << endl;
    return 0;
}
```

### Output

```
count is 3
count is 3
count is 3
```



### Static Member Function

- a member function declared with the static keyword.
- Can be called using the class name without creating an object.
- can access only static data members and other static member functions.
- Can also be called through an object (although calling it through the class name is preferred).

### Example of Static Data and Static Function

```
#include <iostream>
using namespace std;
class Counter{
    private: static int count;
    public: Counter(){
        count++;
    }
    static int getCount(){
        return count;
    }
};
int Counter::count = 0;
int main(){
    Counter c1, c2, c3;
    cout << "Total Objects = " << Counter::getCount();
    return 0;
}
```

#### Output

Total Objects = 3

### Arrays as Class Member Data

```
#include <iostream>
using namespace std;

class Stack //first in last out
{ private: enum { MAX = 10 };
    int st[MAX];
    int top;

public: Stack()
    { top = -1; }

    void push(int var)
    { st[++top] = var; }

    int pop()
    { return st[top--]; }

};
```

```
int main()
{ Stack s1;
    s1.push(11);
    s1.push(22);
    cout << "1: " << s1.pop() << endl; //22
    cout << "2: " << s1.pop() << endl; //11
    s1.push(33);
    s1.push(44);
    s1.push(55);
    cout << "3: " << s1.pop() << endl; //55
    cout << "4: " << s1.pop() << endl; //44
    cout << "5: " << s1.pop() << endl; //33

    return 0;
}
```

### Arrays of Objects

```
#include <iostream>
using namespace std;

class ArrObj
{
    private:    int data;
    public:    void setData(int d){ data = d;    }
               int getData()        { return data; }
};

int main()
{
    ArrObj a[3];  int i;
    for(i=0; i < 3; i++)
        a[i].setData(i);
    for(i=0; i < 3; i++)
        cout << a[i].getData() << "\n";
    return 0;
}
```

### Output

0  
1  
2

### Quiz 1

When **struct** is used instead of the keyword **class** means, what will happen in the program?

- A. Access is public by default
- B. Access is private by default
- C. Access is protected by default
- D. None of the mentioned

Answer is A

### Quiz 2

Like constructors, can there be more than one destructors in a class?

- A. True
- B. False

Answer is B



### Quiz 3

Which of the following is false?

- A. destructors cannot be overloaded
- B. constructors cannot be overloaded
- C. constructors is executed automatically
- D. constructors can have parameters

Answer is B

### Let's summarize what we learned today

#### Constructor

- is invoked automatically when the object created.
- allocates the memory for an object.

#### Destructor

- is invoked when the object goes out of scope.
- deallocate the memory occupied by an object

## Class and Object Practice

1. Write a C++ program to create a class Circle with a data member radius. Create member functions to input the radius and calculate the circumference and area. Use  $\pi = 3.1416$ .
2. Write a C++ program to create a class Rectangle with data members length and width.  
Create:
  - A default constructor and a parameterized constructor to initialize the length and width.
  - A member functions to input the dimensions
  - A member function to calculate the area.

## Class and Object Practice

3. Write a C++ program to create a class Employee with the following data members:

- Employee ID
- Employee Name
- Basic Salary

Create:

- A default constructor that displays "Employee object created."
- A destructor that displays "Employee object destroyed."
- A member function to input the data
- A member function to display the employee information.

## Class and Object Practice

4. Write a C++ program to create a class BankAccount with the following data members:

- Account Number
- Account Holder Name
- Balance

Create member functions to:

- Input account information.
- Deposit an amount.
- Display the account information.

## Class and Object Practice

5. Write a C++ program to create a class named Book with the following data members:

- Book ID
- Title
- Price

Create:

- A default constructor
- A parameterized constructor to initialize the data members.
- A destructor that displays "Book object destroyed."
- A member function to input book information.
- A member function to display book information.
- Display whether the book is Expensive (price  $\geq 50000$ ) or Affordable

## Class and Object Practice

6. Write a C++ program to create a class named ExamResult with the following data members:

- Student Name
- Marks in three subjects (stored in an integer array)

Create member functions to:

- Input student information.
- Calculate the total marks and average.
- Display the student's name, total marks, average, and result. Display Pass if the average is 50 or above; otherwise, display Fail.

Use an array of objects to store information for 5 students.